Doramagic v3 产品定义手册

> 日期: 2026-03-12 状态: 初稿（待 Codex/Gemini 第一性原理批判审查） 演进: v2（22 轮灵魂拷问）→ v3（5 大技术难点深度讨论 + 三方研究） 新增核心输入: Soul Lego Bricks 研究、AI 知识消费格式研究、OpenClaw 多代理模式研究、5 大技术难点结论

---

0. 审查说明

本文档将提交给 Codex 和 Gemini 进行**第一性原理批判审查**。审查者应：

1. **分解到根本事实**：每一条产品定义是否可以从根本事实推导出来？有没有依赖未验证的类比或惯例？
2. **挑战因果链**：A 真的导致 B 吗？有没有跳跃的逻辑？
3. **质疑假设**：哪些假设是"大家都这么做"而非"必须这么做"？
4. **寻找盲区**：有什么重要的事情这份文档没有讨论？

每一节末尾的 `［审查问题］` 是专门留给审查者的思考引导。

---

1. 一句话定义

Doramagic 是运行在 OpenClaw 上的哆啦A梦——用户描述一个模糊的需求，Doramagic 从开源世界提取智慧，在 OpenClaw 中锻造出一个开袋即食的个性化 AI 道具，用户拿到道具就能用，不需要懂任何技术。

关键词解构： – **运行在 OpenClaw 上**：Doramagic 本身是一个 OpenClaw Skill，它产出的道具也是 OpenClaw Skill。OpenClaw 是 Doramagic 的运行时环境，不是可选的部署目标。 – **哆啦A梦**：品牌人设。不教大雄做事，给他道具。 – **模糊的需求**：用户不需要知道该"抄谁的作业"，只需要说出烦恼。 – **从开源世界提取智慧**：核心引擎是 Soul Extractor，提取设计哲学（WHY）和社区踩坑经验（UNSAID）——这是行业空白。 – **锻造**：不是简单拼装，是经过知识编译、质量验证、格式优化的锻造过程。 – **开袋即食**：零配置门槛，交付即可用。 – **个性化 AI 道具**：不是通用工具，是根据用户需求量身定制的专家级道具。

与 v2 定义的区别

	v2	v3
环境定位	"OpenClaw 生态中的"	"运行在 OpenClaw 上的"——从生态参与者变为平台原生应用
技术锚点	多源提取 + 智能装配	Soul Extractor 引擎 + Knowledge Compiler + 子代理架构
新增概念	—	Soul Lego Bricks、Knowledge Compiler、确定性编排层、置信度分级
用户旅程	概念性描述	完整定义"发明工具"和"使用工具"两条旅程

> [审查问题] "运行在 OpenClaw 上"的定位是否过度绑定？如果 OpenClaw 平台发生重大变化，Doramagic 的核心价值是否还能独立存在？

---

2. 产品哲学

2.0 产品设计之魂（不可修改）

> **不教用户做事，给他工具。**
>
> 这是 Doramagic 的产品设计之魂。哆啦A梦从不教大雄怎么做事——他从口袋里掏出道具，交给大雄，让大雄自己去解决问题。

这条哲学约束产品设计的每一个决策。任何需要用户"学习使用方法"的设计都违反了这条哲学——道具应该自解释，而不是附带说明书。

2.1 知识魔法 vs 方法论魔法

> Doramagic 是知识魔法（注入能力），不是方法论魔法（强加流程）。

方法论 Skill 越严格越容易被删除（因为它约束用户）；知识 Skill 越丰富越被依赖（因为它赋能用户）。Doramagic 生成的道具不是固定 stage 流水线，而是知识驱动的自适应行为——AI 自己判断什么时候走完整流程，什么时候直接干。

第一性原理验证：LLM 是语言模型，其能力本质是理解和生成语言。给 LLM 注入知识（叙事体的设计哲学、因果关系的决策规则、案例形式的踩坑经验）让它"懂行"，比给它一套固定步骤让它"执行"效果好得多。

这已被实验验证（exp01: 42%→96%）。

2.2 操作化 > 知识增量

> Doramagic 的价值 = LLM 已有知识 × 操作化基础设施

LLM 已经"知道"很多领域知识。Doramagic 的价值不在于补充 LLM 不知道的那 10%，而在于将 LLM 已有的知识**操作化**——连接真实 API、注入特定数据源、注入经过验证的领域决策规则、添加社区踩坑经验。正如 Claude（模型）知道怎么写代码，而 Claude Code（产品）加上文件系统、Git、工具编排，将编程知识产品化为 10 倍有用的东西。

2.3 抄作业高手，非从零构建

核心能力是**选择**和**适配**，而非**创造**。Doramagic 不发明、不创造——发现最好的现有方案并为用户量身适配。成功取决于"可抄的好作业"的丰富程度，而非 AI 的创造力。

2.4 有立场的专家，非信息搬运工

大多数 AI 产品不敢做判断——列出 A/B/C 三个选项说"取决于你的需求"。这是推卸责任。用户要的不是零件清单，是成品。成品意味着有人替你做了判断。Doramagic 必须替用户做出选择。

2.5 代码说事实，AI 说故事

这是 Doramagic 的架构第一性原则，贯穿整个技术栈：

- **事实层**：由确定性脚本提取（文件结构、依赖关系、API 签名、命令列表），零模型自由度
- **解读层**：由 LLM 生成（设计哲学、心智模型、决策理由、社区经验），有约束的模型自由度

事实不依赖模型，故事允许有风格差异。这一原则同时解决了幻觉问题（事实层不可能幻觉）和跨模型一致性问题（事实层零波动）。

> [审查问题] "代码说事实，AI 说故事"——在实践中，"事实"和"故事"的边界是否真的清晰？有没有本质上模糊的中间地带？

---

3. OpenClaw 环境定位

3.1 为什么 OpenClaw First

从第一性原理推导：

```
Doramagic 的输出是"AI 道具"
  → AI 道具需要运行时环境（模型调用、工具编排、记忆持久化、用户交互）
    → 自建运行时 = 重新发明 OpenClaw
      → OpenClaw 已经解决了这些基础设施问题
        → Doramagic 应该站在 OpenClaw 上，而非旁边
```

OpenClaw 提供的基础设施：

OpenClaw 能力	Doramagic 如何利用
Gateway + Sessions	Doramagic 本身作为 Skill 运行在 session 中；生成的道具也运行在 session 中
50+ Channels	用户通过任意渠道（WhatsApp/Telegram/WebChat 等）触发 Doramagic 和使用道具
内置工具（exec/read/write/browser/web_search）	Soul Extractor 在提取过程中使用这些工具
MCP Server 支持	生成的道具通过 MCP 协议接入外部资源
Skills 生态（5400+）	Doramagic 生成的道具可调用其他已安装 Skill
Subagents（sessions_spawn）	Soul Extractor 使用子代理进行多阶段并行提取
MEMORY.md	道具运行记忆 + 用户偏好持久化 + 反馈闭环
多模型路由	不同阶段使用不同模型（Opus/Sonnet/MiniMax），优化成本和质量

3.2 Doramagic 在 OpenClaw 中的身份

```
OpenClaw 平台
├── 内置工具（exec, read, write, browser, ...）
├── 已安装 Skills（5400+ from ClawHub）
│   ├── trello
│   ├── himalaya（邮件）
│   ├── brave-search
│   ├── ...
│   └── ★ doramagic                    ← Doramagic 本身是一个 Skill
│       ├── SKILL.md                    ← 主入口
│       └── skills/                     ← Doramagic 内部的技能资产
│           ├── soul-extractor/         ← 灵魂提取引擎
│           └── skill-forge/            ← 道具锻造器
├── 用户生成的 Skills
│   ├── my-stock-tracker               ← Doramagic 锻造的道具
│   ├── my-fitness-coach               ← Doramagic 锻造的道具
│   └── ...
└── MCP Servers
```

Doramagic 是 OpenClaw 的**道具工厂**——它本身是一个 Skill，它的产出也是 Skill。

3.3 OpenClaw 的约束（硬性）

必须适应，不可改变：

约束	含义	Doramagic 的适配策略
SOUL.md 加载不可靠	模型可能忽略 SOUL.md 中的指令	关键知识写入 Skill 主体（SKILL.md content 区），不依赖 SOUL.md
子代理不继承 SOUL.md	子代理只拿 AGENTS.md + TOOLS.md	每个子代理的指令必须显式传入，不依赖全局配置
默认模型为 MiniMax	不指定模型时使用 MiniMax，能力有限	关键阶段（WHY 提取、叙事合成）必须显式指定 Opus/Sonnet
上下文窗口有限	单次对话的 token 预算有限	子代理架构解决——每个阶段独立上下文，不互相挤压
Skill 文件结构固定	SKILL.md + frontmatter + content	生成的道具必须严格遵循 OpenClaw Skill 格式
异步执行	子代理异步运行，无法实时监控	确定性检查点，每个阶段有质量门控

> [审查问题] 这些约束是 OpenClaw 2026.3 版本的约束。随着 OpenClaw 演进，这些约束可能放松或改变。Doramagic 的架构是否有足够的弹性来适应平台变化？

---

4. 用户旅程一：发明工具

> 用户通过 Doramagic "发明"一个个性化 AI 道具的完整过程。

4.1 旅程总览

```
用户："Dora，帮我做一个能追踪 A 股行情的东西"
                    │
                    ▼
          ┌──────────────────────┐
          │  Phase 1: 需求挖掘    │  苏格拉底对话，3-5 轮
          │  "你最想解决什么？"    │  产出：需求画像
          └──────────────────────┘
                    │
                    ▼
          ┌──────────────────────┐
          │  Phase 2: 发现作业    │  搜索开源项目 + MCP + Skill 市场
          │  "找到了 3 个好作业"   │  产出：源泉地图
          └──────────────────────┘
                    │
                    ▼
          ┌──────────────────────┐
          │  Phase 3: 提取灵魂    │  Soul Extractor 引擎工作
          │  "正在提取智慧..."    │  产出：知识卡片 + 置信度
          └──────────────────────┘
                    │
                    ▼
          ┌──────────────────────┐
          │  Phase 4: 锻造道具    │  Knowledge Compiler + 合成
          │  "为你量身定制中..."   │  产出：可运行的 Skill 包
          └──────────────────────┘
                    │
                    ▼
          ┌──────────────────────┐
          │  Phase 5: 交付验证    │  冒烟测试 + 安装
          │  "道具准备好了！"     │  产出：已安装的道具
          └──────────────────────┘
```

4.2 Phase 1: 需求挖掘

触发方式： – `/dora 我想追踪A股行情` – "Dora，我需要一个能帮我选股的东西" – "帮我做一个 xxx"

Doramagic 的行为（苏格拉底对话，每轮 1–2 个问题）：

轮次	目的	示例
R1	问题本质	"你想让 AI 帮你做什么？最痛的一件事是什么？"
R2	场景与频率	"你多久需要看一次行情？现在怎么做的？"
R3	成功标准	"做到什么程度你会觉得'这真好用'？"
R4	偏好与约束	"你更想要简单快速的还是功能全面的？有没有已经在用的数据源？"
R5	确认	"我理解你需要一个 A 股实时行情追踪器，核心功能是 X，对吗？"

硬门控：需求画像必须包含以下字段，否则继续对话： – `core_problem`： 核心问题是什么 – `trigger`： 什么时候需要用这个工具 – `expected_output`： 期望得到什么样的输出 – `user_environment`： 用户的 OpenClaw 环境（已安装 Skill、可用模型、渠道）

环境感知：Doramagic 在对话开始前自动读取用户的 OpenClaw 环境： – 已安装了哪些 Skill（避免重复造轮子） – 可用的 MCP Server（知道能连接什么资源） – 用户的 MEMORY.md（了解历史偏好）

4.3 Phase 2: 发现作业

Doramagic 在用户等待时，自动执行多源搜索：

四梯队来源模型：

梯队	来源	价值定位	搜索方式
T1	MCP Server 注册表 + GitHub 仓库 + OpenClaw Skill 市场	核心能力来源	web_search + ClawHub API
T2	API 文档（OpenAPI） + Stack Overflow + Awesome-lists	操作知识补充	web_search + web_fetch
T3	技术博客 + 领域垂直社区	经验知识	web_search
T4	会议演讲 + 论坛讨论	长尾覆盖	未来阶段

产出：源泉地图（source_map），标注每个来源的价值定位和优先级。

用户交互：向用户展示发现结果，请求确认或补充。

> "我找到了 3 个与你需求高度相关的开源项目：akshare（A 股数据接口）、mootdx（通达信行情）、quantaxis（量化框架）。你想让我从哪些项目中提取智慧？"

4.4 Phase 3: 提取灵魂

这是 Doramagic 的核心竞争力所在——Soul Extractor 引擎工作。

灵魂的六个知识层（v3 新增重点标注）：

知识层	含义	提取方式	竞品覆盖度
领域知识（该知道什么）	概念、理论、设计哲学	LLM 推理 + 积木锚定	部分
操作知识（怎么做）	API 调用、工具配置	确定性脚本 + LLM 补充	有
经验知识（什么有坑）	社区踩坑、已知陷阱	Issues/讨论/博客交叉验证	弱
决策知识（何时用什么）	选型判断、权衡规则	LLM 推理 + 社区证据	极弱
★ **设计哲学**（为什么这样）	WHY、心智模型、架构意图	LLM 第一性原理推理	**行业空白**
★ **隐性知识**（没人说的事）	UNSAID、暗默假设、反模式	社区信号 + LLM 推断	**行业空白**

后两层（WHY + UNSAID）是 Doramagic 的护城河。没有任何现有工具能自动提取设计哲学和隐性知识。

提取架构（v3 新增——基于难点 3 结论）：

```
确定性编排层（Shell 脚本 / OpenClaw exec 工具）
├── Stage 0: 确定性提取
│   ├── repo_facts.json（文件结构、依赖、命令、API 签名）
│   ├── 框架检测 → 加载对应灵魂积木
│   └── ⋆ 依赖热力图（识别 integration_hub 脉络点）
│
├── Stage 1: WHY 提取 ← 子代理 model: Opus
│   ├── 输入: 积木(≤2500) + 代码片段 + 热力图
│   ├── 约束: 第一性原理推理，每条 WHY 附推理链
│   └── 产出: WHY 卡片 + confidence A/B/C/D
│
├── Stage 2-3: 概念/规则提取 ← 子代理 model: Sonnet + 积木锚点
│   ├── 输入: 积木锚点(≤1000) + 代码片段
│   └── 产出: 概念卡 + 规则卡 + confidence
│
├── Stage 3.5: 验证关卡 ← 确定性脚本
│   ├── fact-checking gate（事实 vs repo_facts.json）
│   ├── 置信度标注（A/B/C/D 分级）
│   └── 社区证据交叉验证
│
├── Knowledge Compiler ← 确定性编译
│   ├── 按知识类型路由格式:
│   │   事实型 → 结构化列表
│   │   规则型 → 规则语句 + 因果注释
│   │   哲学型/踩坑型 → 纯叙事
│   ├── 按置信度过滤（A/B 纳入，C 降为注释，D 丢弃或标注）
│   └── token 预算控制（≤2000 tokens）
│
└── Stage 4: 叙事合成 ← 子代理 model: Opus
    ├── 输入: 编译后的卡片 + SKILL.md 模板
    └── 产出: 完整的 Skill 包
```

关键设计决策解释：

1. **为什么用子代理而非单一 agent?** — 每个阶段独立上下文。Stage 1 只需要积木+代码（~18K tokens），Stage 4 只需要卡片+模板（~6K tokens）。不存在上下文预算竞争。（参见难点 4 结论）

2. **为什么 WHY 提取用 Opus?** — WHY 层知识需要深度推理（从代码结构推断设计意图），这是 Opus 的强项。概念/规则提取相对直接，Sonnet 足够。

3. **为什么有 Knowledge Compiler?** — 源格式（结构化卡片）≠ 消费格式（面向 AI 的叙事/规则混合体）。结构化用于存储和验证，叙事化用于 AI 消费。"结构化抽取，叙事化编译，分块式注入。"（参见难点 2 三方研究结论）

4. **为什么编排层是确定性的?** — Prompt 是建议不是约束。把关键步骤从模型职责中拿走，放到确定性编排层，模型只负责"理解和生成"——这正是 LLM 擅长的。（参见难点 3 结论）

4.5 Phase 4: 锻造道具

将提取的知识编译成一个可运行的 OpenClaw Skill。

产出结构（严格遵循 OpenClaw Skill 格式）：

```
# 生成的 Skill 文件结构
my-stock-tracker/
├── SKILL.md              # 主文件（frontmatter + 知识注入的内容）
├── references/           # 参考文档（可选）
│   └── api-reference.md  # API 使用说明
└── _meta.json            # ClawHub 元数据
```

SKILL.md 的内容结构（基于 AI 知识消费格式研究）：

```
---
name: my-stock-tracker
description: >
  A 股行情追踪与分析助手。
  Triggers on: "行情", "股票", "A股", "涨跌", "stock"
user-invocable: true
metadata:
  openclaw:
    category: finance
    requires:
      bins: []
---

# A 股行情追踪助手

[开头——高注意力区]
## 你是谁
一句话定位 + 心智模型类比

## 设计哲学
纯叙事，2-3 个因果链（来自 WHY 提取）

## 关键规则
≤5 条，每条 = 规则 + 一行因果注释（来自规则提取 + Knowledge Compiler）

[中部——用叙事缓解注意力衰减]
## 踩坑经验
2-3 个案例故事（来自社区信号 + UNSAID 提取）

[尾部——高注意力区]
## 能力清单
结构化列表——能做什么、用什么 API、有什么限制（来自 repo_facts 确定性生成）

## 速查表
≤8 行 Markdown 表格
```

这个结构遵循 Lost in the Middle 的 U 型注意力分布——关键知识放开头和结尾（高注意力区），中间用叙事体缓解注意力衰减。总预算 1500–2000 tokens。

4.6 Phase 5: 交付验证

1. **冒烟测试**：验证 Skill 格式正确、资源可达、触发词有效
2. **安装**：将 Skill 安装到用户的 OpenClaw workspace
3. **首次使用引导**：Doramagic 告诉用户触发词，演示一次调用

> "你的 A 股行情追踪助手已经准备好了！以后直接说'帮我看一下今天 A 股行情'就能用。要不要现在试一下？"

4.7 用户可见的进度

> "足够透明：让用户看到生产过程的关键进展，而非等 30 分钟得到一个黑盒结果。"

每个 Phase 完成时向用户汇报： – Phase 1 完成："明白了你的需求，开始找好作业" – Phase 2 完成："找到了 3 个相关项目，开始提取智慧" – Phase 3 完成："智慧提取完成，开始锻造道具" – Phase 4 完成："道具锻造完成，正在做最后的测试" – Phase 5 完成："一切就绪！"

> [审查问题] Phase 3（灵魂提取）可能是最耗时的阶段。用户等待时间如何管理？是否需要更细粒度的进度反馈？如果提取耗时超过 10 分钟，用户是否会流失？

---

5. 用户旅程二：使用发明的工具

> 用户拿到 Doramagic 锻造的道具后，如何使用它、如何与它互动、如何让它变得更好。

5.1 核心原则：道具自解释

> **不教用户做事，给他工具。** 这条哲学在"使用"阶段的推论是：**道具本身必须足够自解释，用户不需要阅读说明书。**

实现方式： – **触发词自然**：道具的 trigger 就是用户日常会说的话（"帮我看看行情"而不是"/run stock-tracker"） – **首次使用引导极简**：Doramagic 交付时只说一句话——告诉用户触发词 – **道具自我介绍**：用户首次触发道具时，道具用一句话介绍自己的能力范围 – **渐进暴露**：基础功能直接可用，高级功能在用户需要时自然浮现

5.2 道具的行为特征

Doramagic 锻造的道具不是固定流程执行器，而是**注入了领域知识的 AI 专家**。

特征	通用 AI	Doramagic 道具
"帮我分析茅台"	会给一段网上都能找到的分析	知道用 akshare 拿实时数据，知道 PE/PB 多因子分析的正确做法，知道行业常见的数据陷阱
遇到边界情况	"这取决于你的需求"	有明确立场，基于提取的决策规则做判断
犯错概率	高（依赖模型通用知识）	低（有验证过的领域知识锚定，减少幻觉面）
使用方式	每次需要重新描述上下文	知识已内化，自动应用

5.3 渠道适配

因为道具运行在 OpenClaw 上，所以它天然支持 OpenClaw 的所有渠道： – 微信/WhatsApp：在聊天中直接使用 – Telegram：群聊中多人共用 – WebChat：浏览器访问 – 桌面/移动端：通过 OpenClaw 客户端

用户在哪里使用 OpenClaw，就在哪里使用 Doramagic 的道具。道具不需要自己的 UI。

5.4 道具进化：回炉改造

道具不是一次性产品。用户使用过程中，道具通过 MEMORY.md 积累使用数据。当用户觉得道具不够好时：

```
用户："Dora，帮我改一下我的行情追踪器，我希望它能同时看港股"
                        │
                        ▼
Doramagic 读取原道具 + 使用记忆
                        │
                        ▼
增量提取（只补充港股相关知识，不重新从零提取）
                        │
                        ▼
重新编译 + 安装
                        │
                        ▼
道具 v1.0 → v1.1
```

关键设计： – **增量提取**：不重新从零提取，基于已有知识做增量补充 – **用户触发**：不是自动优化，是用户主动要求 – **保留学习记录**：每次改造的原因和变更都记录在 MEMORY.md 中

5.5 道具之间的协作

用户可能有多个 Doramagic 锻造的道具。这些道具可以： – **互相调用**：行情追踪器可以调用选股器 – **共享知识**：同一领域的道具共享领域知识（通过 MEMORY.md） – **组合使用**：用户自然地在对话中同时使用多个道具

这依赖 OpenClaw 的 Skill 间调用能力和共享 workspace。

> [审查问题] 道具之间的"共享知识"在实现上如何做到？MEMORY.md 是 workspace 级别的还是 Skill 级别的？如果多个道具写同一个 MEMORY.md，冲突如何解决？

---

6. Soul Extractor 引擎：技术核心

6.1 灵魂积木（Soul Lego Bricks）

概念：预提取的框架级知识，作为 Soul Extractor 的参考原材料。

从第一性原理推导：

```
框架级知识（如 Django ORM 的设计哲学）是公共知识
  → 每次提取都让 LLM 从头推理，浪费 + 容易幻觉
    → 预提取一次，验证后反复使用
      → LLM 有了"锚点"，提取精度提升，幻觉面缩小
```

积木规格（基于三方研究结论）： – L1 框架级积木：200–400 tokens（如 django–core），覆盖框架核心哲学 – L2 模式级积木：400–800 tokens（如 django–orm），覆盖特定模式的设计意图 – 总预算：≤2500 tokens（注入子代理上下文时不挤压代码空间）

积木的双重作用： 1. **锚定**：有了框架积木，提取引擎知道哪些是"公共知识"（已覆盖），只提取项目特有的增量知识 2. **过滤**：偏离框架核心设计理念太远的提取结果，大概率是噪音

6.2 三层幻觉防护

层	时机	机制
第一性原理约束	提取时	逼模型从代码出发推理，每条 WHY 附推理链
社区证据交叉验证	验证时	用 Issues/RFC/博客找支持/反驳证据 → 置信度 A/B/C/D
项目规模调节策略	元策略	高 Star 项目警惕记忆污染+强化交叉验证，低 Star 项目更依赖代码推理链

置信度分级： – **A**：创作者明确说明（README/文档/注释中直接表述） – **B**：社区证据支持（Issues/讨论/博客中有相关讨论） – **C**：代码模式推断（从代码结构推断但无外部证据） – **D**：纯 LLM 推理（模型自行推断，无代码或社区支持）

6.3 Knowledge Compiler

源格式 ≠ 消费格式。Knowledge Compiler 是连接提取和消费的桥梁。

```
结构化源卡（CC/WF/DR）
    │
    ▼
Knowledge Compiler
    │
    ├── 路由：按知识类型选择格式
    │   ├── 事实型 → 结构化列表
    │   ├── 规则型 → 规则语句 + 因果注释
    │   ├── 工作流型 → 编号列表 + 叙事说明
    │   ├── 哲学型 → 纯叙事
    │   └── 踩坑型 → 案例故事
    │
    ├── 过滤：按置信度筛选
    │   ├── A/B → 纳入
    │   ├── C → 降为注释
    │   └── D → 丢弃或标注
    │
    ├── 压缩：控制 token 预算
    │   └── 总预算 ≤2000 tokens
    │
    └── 编译：按 Lost in the Middle 原则排列
        ├── 开头：哲学 + 关键规则（高注意力区）
        ├── 中间：踩坑故事（叙事缓解注意力衰减）
        └── 尾部：能力清单 + 速查表（高注意力区）
```

6.4 依赖热力图（v3 新增）

Stage 0 确定性分析产出依赖热力图，识别系统的**脉络点**——被多处引用的核心集成点。

```
{
  "heatmap": [
    {"node": "core.api.authenticate()", "references": 14, "type": "integration_hub"},
    {"node": "utils.cache.get()", "references": 8, "type": "infrastructure"},
    {"node": "helpers.format_date()", "references": 2, "type": "utility"}
  ]
}
```

脉络点的价值：被 14 处引用的 API 远比只被 2 处使用的 helper 更值得深度提取。热力图注入给后续子代理，指导提取优先级。

> [审查问题] 依赖热力图是否足以捕捉所有类型的"脉络点"？有没有非代码依赖的脉络（如共享概念、隐式约定）无法通过静态分析发现？

7. 技术架构

7.1 架构全景

```
用户（通过 OpenClaw 任意渠道）
        │
        ▼
┌──────────────────────────────────────────────────────┐
│  Doramagic Skill（SKILL.md）                          │
│                                                       │
│  确定性编排层                                          │
│  ├── Phase 路由（需求→发现→提取→锻造→交付）            │
│  ├── 质量门控（每阶段硬门控）                          │
│  └── 子代理调度（sessions_spawn）                      │
│                                                       │
│  子代理层                                              │
│  ├── 需求 Agent（Sonnet）  — 苏格拉底对话              │
│  ├── 发现 Agent（Sonnet）  — 多源搜索                  │
│  ├── 提取 Agent ×N（Opus/Sonnet）— 并行提取            │
│  ├── 锻造 Agent（Opus）   — 叙事合成                   │
│  └── 验证 Agent（Sonnet）— 冒烟测试                    │
│                                                       │
│  确定性工具层                                          │
│  ├── prepare-repo.sh     — git clone + 打包            │
│  ├── extract_repo_facts.py — 确定性事实提取            │
│  ├── knowledge_compiler  — 格式编译                    │
│  ├── assemble-output.sh  — 最终组装                    │
│  └── validate_output.py  — 质量验证                    │
│                                                       │
│  知识资产层                                            │
│  ├── Soul Lego Bricks（预提取的框架级知识）            │
│  └── 积木仓库（持续扩展）                              │
└──────────────────────────────────────────────────────┘
        │
        ▼
用户 workspace 中安装的道具（OpenClaw Skill）
```

7.2 模型路由策略

阶段	模型	理由
编排/调度	无（确定性脚本）	关键步骤不依赖模型
需求对话	Sonnet	对话理解足够
源泉发现	Sonnet	搜索和筛选不需要深度推理
WHY/UNSAID 提取	Opus	需要深度推理能力
概念/规则提取	Sonnet + 积木锚点	有积木辅助，Sonnet 足够
叙事合成	Opus	高质量叙事需要深度语言能力
冒烟测试	Sonnet	验证任务不需要深度推理

7.3 MCP-first 资源层

生成的道具通过 MCP 协议接入外部资源：

```
原方案: Skill 内嵌 import akshare; ak.stock_zh_a_spot_em()
升级方案: Skill 通过 MCP Client 调用 → mcp://stock-data/realtime?market=A
          → 背后对接 akshare MCP Server（可独立更新/替换）
```

好处： – 资源可独立更新（MCP Server 升级不影响道具） – 可跨道具复用（多个道具共用同一个 MCP Server） – 符合 OpenClaw 原生集成模式

7.4 上下文预算管理

子代理架构天然解决上下文预算竞争——每个阶段独立上下文：

阶段	上下文内容	预算	不需要
Stage 1 (WHY)	积木(≤2500) + 代码片段(≤20K)	~23K	CLAUDE.md
Stage 2-3 (概念/规则)	积木锚点(≤1000) + 代码片段(≤15K)	~16K	CLAUDE.md
Stage 4 (合成)	编译后卡片(≤5K) + SKILL.md 模板(≤500)	~6K	项目代码

没有任何一个阶段需要同时塞入全部知识。这是调度问题，不是预算问题。

> [审查问题] 上下文预算估算是否经过实测验证？"代码片段 ≤20K"是否足以覆盖大型项目的关键代码？如果不够，选择哪些代码片段的策略是什么？

8. 质量保证体系

8.1 四层质量门控

门控	阶段	检查内容	失败处理
需求完整性	Phase 1 后	need_profile 包含全部必填字段	继续对话
源泉充分性	Phase 2 后	≥1 主力源泉 + ≥1 资源源泉	回退到 Phase 1 补充搜索词
提取质量	Phase 3 后	≥3 知识卡（A/B 级）+ 事实验证通过	换源泉或重试
道具可用性	Phase 5	Skill 格式正确 + 触发词有效	回退到 Phase 4 修复

8.2 事实验证

- `repo_facts.json` 白名单验证：提取的命令、API、配置项必须在白名单中存在
- 不在白名单中的事实标记为"未验证"，不纳入能力清单
- 这是 P0 补丁（已实现并验证）的产品化

8.3 置信度透传

从提取到最终道具，置信度信息全程透传： – 提取阶段：每张卡片标注 confidence + evidence_source – 编译阶段：按置信度筛选（A/B 纳入，C 注释，D 丢弃） – 道具内部：知识以"已验证"和"推断性"区分呈现

8.4 提取噪音控制

提取噪音四类及解法：

噪音类型	危害	解法
幻觉（错误信号）	致命	三层防护
不相关事实（低信号）	高	积木减少搜索面 + 依赖热力图指导优先级
冗余信息	中	Knowledge Compiler 去重压缩
过度解释	中	token 预算硬约束

> [审查问题] "不相关但正确"的知识如何定义？对用户 A 不相关的知识可能对用户 B 很重要。个性化需求如何在提取阶段就纳入考量？

---

9. 护城河

9.1 核心护城河：WHY + UNSAID 提取能力

竞品分析结论（GLM5 交叉验证）：

工具	覆盖层	智慧层覆盖
Repomix	代码结构（WHAT）	无
Cursor Rules	行为规则（HOW）	无
Graph4Code	代码关系图（WHAT）	无
GPTMe	对话式编程（HOW）	无
Doramagic	**全六层**	**★ WHY + UNSAID = 行业空白**

没有任何现有工具能自动提取设计哲学、决策理由和隐性知识。Doramagic 的 Soul Extractor 是唯一瞄准"智慧层"的工具，v1.0 已验证可行（42%→96%）。

9.2 飞轮效应

```
积木仓库持续扩展
  → 提取精度持续提升（更好的锚点 = 更少的幻觉）
    → 道具质量持续提升
      → 用户满意度提升
        → 更多使用 = 更多反馈
          → 道具持续进化
            → 吸引更多用户
              → 更多需求 = 更多框架积木需求
                → 积木仓库持续扩展 ...
```

9.3 价值公式

> 帮低级用户从 30 分到 60 分，帮高级用户从 80 分到 85 分。

用户类型	输入质量	Doramagic 价值	典型场景
老王（散户）	"帮我炒股"	30 → 60 分（巨大 delta）	得到一个真正懂 A 股数据的 AI 助手
专业用户	"基于 PE/PB 多因子选股"	80 → 85 分（精细 delta）	得到一个注入了 akshare 最佳实践的精准工具

10. 品牌与交互

10.1 品牌映射

哆啦A梦	Doramagic
大雄带着烦恼来找哆啦A梦	用户带着模糊需求触发 /dora
哆啦A梦理解真实需求	苏格拉底对话发现真实问题
从四次元口袋里掏出道具	从开源世界提取智慧，锻造道具
道具交给大雄，大雄自己用	道具安装到 OpenClaw，用户直接用
道具被滥用时哆啦A梦警告	透明展示能力边界 + 置信度
大雄遇到新问题再来找	回炉改造

10.2 触发方式

- `/dora` + 需求描述
- "Dora，帮我 XXX"
- 自然语言触发（description 中定义）

10.3 人设

亲切、直接、偶尔吐槽。不教育用户，不炫技术。像一个靠谱的朋友——你说了一句话，他就懂你要什么，然后默默帮你搞定。

---

11. 开放问题与风险

11.1 已明确的开放问题

问题	当前状态	解决路径
输出形态	确定为 OpenClaw Skill 格式	v3 已明确
提取质量	v1.0 验证可行（42%→96%），但仅单源	exp09 验证多源+积木+新格式
装配质量	未验证	exp09 端到端实验
积木仓库冷启动	未开始	先做 django-core + django-orm 两块积木
OpenClaw 子代理实际能力	部分确认（支持模型路由、最多 5 并发）	实现阶段验证 Bash 工具可用性、中间产物传递

11.2 核心风险

风险	来源	严重度	缓解
提取质量不够锻造可用道具	最危险的假设	致命	积木+三层防护+置信度+exp09 验证
OpenClaw 平台约束过严	SOUL.md 不可靠等	高	适配策略已明确，关键知识写 SKILL.md 主体
模型依赖性	MiniMax 质量差	高	关键阶段显式指定 Opus/Sonnet
用户等待时间	端到端可能耗时 5-15 分钟	中	细粒度进度反馈 + 异步执行
积木维护成本	框架更新需要积木更新	中	知识保鲜期以年计（Tang 判断），可控

11.3 MVP 验证路径

```
Step 1: 积木冷启动
  → 制作 django-core + django-orm 积木
  → 验证积木对提取质量的提升

Step 2: Soul Extractor v2
  → 子代理架构 + Knowledge Compiler + 置信度
  → 在已有 benchmark 上验证

Step 3: 端到端验证
  → 从 "帮我做一个 Django 管理后台" 到生成可用 Skill
  → 在 OpenClaw 中实际运行

Step 4: 跨领域验证
  → 从 Django 扩展到其他领域（如金融/健身）
  → 验证积木+引擎的通用性
```

---

12. 与 v1.0 / v2.0 对比

维度	v1.0	v2.0	v3.0
定义	灵魂提取工具	道具制造器	OpenClaw 上的哆啦A梦
环境	CLI	OpenClaw 生态	OpenClaw 原生（First）
入口	GitHub URL	一句话需求	一句话需求（+ 环境感知）
出口	CLAUDE.md	可运行的 Skill	OpenClaw Skill（+ 置信度 + 溯源）
提取架构	单 LLM 串行	Orchestrator + 子代理	确定性编排 + 模型路由子代理
幻觉防护	无	repo_facts.json	三层防护 + 置信度 A/B/C/D
知识编译	无	概念性描述	Knowledge Compiler（按类型路由格式）
框架知识	每次从头推理	概念性提及	Soul Lego Bricks（预提取 + 锚定 + 过滤）
上下文管理	全量塞入	分块注入	子代理独立上下文（调度解决竞争）
跨源溯源	无	概念性描述	标签（纵向）+ 脉络（横向）+ 热力图
用户旅程	未定义	概念性描述	完整定义"发明"和"使用"两条旅程

附录 A: 5 大技术难点及解决方案索引

难点	核心问题	解决方案	详细记录
1. 幻觉与证据绑定	WHY 层知识不可验证	三层防护 + 置信度 A/B/C/D	memory/doramagic-5-challenges.md
2. 确定性与生成式边界	"代码说事实，AI说故事"的落地	Knowledge Compiler + 按类型路由格式	+ synthesis-report.md
3. 跨模型/Agent 控制	模型跳过脚本、质量波动	确定性编排层 + 子代理模型路由	+ OpenClaw 多代理研究
4. 知识密度与可用性	提取噪音 + 上下文竞争	积木过滤 + 子代理调度	+ AI 知识消费研究
5. 多源融合与端到端	冲突解决 + 溯源 + 组装	可信度链 + 标签脉络 + 前 4 方案集成	—

附录 B: 硬性约束清单

以下约束不可被产品迭代或技术决策推翻：

1. **第一性原理**：所有技术决策必须从根本事实推导，不依赖类比或惯例
2. **产品设计之魂**：不教用户做事，给他工具
3. **代码说事实，AI 说故事**：事实层零模型自由度
4. **OpenClaw 适配**：Doramagic 适应平台，不可反过来
5. **Knowledge Compiler 不增删内容**：只做格式转换，编译规则严格

附录 C: 关键三方研究索引

研究	路径	核心结论
灵魂积木粒度	research/soul-lego-bricks/reports/synthesis-report.md	2 层（L1 框架 200-400 + L2 模式 400-800），总 ≤2500 tokens
AI 知识消费格式	research/ai-knowledge-consumption/reports/synthesis-report.md	"结构化抽取，叙事化编译，分块式注入。按知识类型路由格式。"
OpenClaw 多代理	memory/doramagic-5-challenges.md（难点 3）	Superpowers SDD 模式 + OpenClaw 子代理能力确认